CISC-8 Processor

Requirements Specification

8-bit CISC processor with Harvard architecture,
windowed register file and native I^2C controller

Date	8 November 2026
Author	Michal Lagosz

Contents

1 Introduction **3**

1.1 Conventions . . . 3

2 General architecture and memory **3**

2.1 General architecture . . . 3

2.2 Program memory . . . 4

2.3 Data memory . . . 4

3 Windowed register file **4**

3.1 Mapping . . . 4

3.2 Calling convention . . . 5

3.3 Requirements . . . 5

4 Special function registers and flags **6**

4.1 SFR map . . . 6

4.2 Status register SR . . . 6

4.3 Requirements . . . 6

5 Instruction set architecture (ISA) **7**

5.1 Instruction formats . . . 7

5.2 Addressing modes . . . 8

5.3 Two-operand operations . . . 8

5.4 Other instructions . . . 8

5.5 Requirements . . . 9

6 I^2C controller **10**

6.1 Rationale for the register set . . . 10

6.2 Registers . . . 10

6.3 High-level instructions . . . 11

6.4 Requirements . . . 12

7 Interrupts, exceptions and reset **12**

7.1 Vector table . . . 13

7.2 Requirements . . . 13

8 External interfaces and non-functional requirements **13**

8.1 Core signals . . . 14

8.2 Interface requirements . . . 14

8.3 Non-functional requirements 14

List of Figures

1 Register windows: 16 blocks of 4 registers, adjacent windows share one block . . . 5

2 Encoding of a two-operand instruction (opcode = OP · 8 + MODE) 7

3 I²C frame generated by `I2CRR`: register number write, repeated START, single-byte read 12

1 Introduction

This document specifies the requirements for CISC-8: an 8-bit CISC processor with a Harvard architecture, 64 general-purpose registers accessed through a sliding window of 8 registers, and a hardware I^{2}C controller. It is the baseline for the RTL design.

1.1 Conventions

Every requirement has a unique identifier of the form `PREFIX-NNN` and is verifiable. The following keywords define the weight of a requirement:

Keyword	Meaning
MUST / MUST NOT	Mandatory; failure to comply is a non-conformance.
SHOULD	Recommended; any deviation must be justified in the design documentation.
MAY	Optional; left to the implementer.

Prefixes: `ARCH` (general architecture), `MEM` (memory), `REG` (registers), `SFR` (special function registers), `ISA` (instructions), `I2C` (I^{2}C controller), `INT` (interrupts and reset), `IO` (interfaces), `NF` (non-functional).

Hexadecimal numbers are written with the prefix `0x`, binary numbers with `0b`. Bit 0 is the least significant bit (LSB).

2 General architecture and memory

The processor is 8-bit (ALU, registers and memory words are 8 bits wide), with 16-bit addresses for program and data in separate 64 KiB address spaces.

2.1 General architecture

ID	Requirement
`ARCH-001`	The processor **MUST** be 8-bit: the ALU, the general-purpose registers and the data buses are 8 bits wide.
`ARCH-002`	The processor **MUST** use a Harvard architecture: separate program and data memories with independent address and data buses.
`ARCH-003`	The processor **MUST** implement a CISC instruction set: variable-length instructions (1–4 bytes) and arithmetic/logic instructions with an operand in data memory (as source or destination).
`ARCH-004`	The processor **MUST** operate in a single clock domain, synchronous to the rising edge of `CLK`.
`ARCH-005`	Instruction execution **MUST** proceed through the phases fetch, decode, operand fetch, execute and write-back. The cycle count of every instruction **MUST** be deterministic and listed in the instruction timing table, except for I^{2}C instructions that wait for the bus (`ISA-040`).
`ARCH-006`	The control unit **MUST** be implemented as hard-wired control based on a finite state machine (FSM), with one or more states per execution phase of each instruction.
`ARCH-007`	16-bit values in memory and in register pairs **MUST** be stored little-endian (low byte at the lower address / in the lower-numbered register).

2.2 Program memory

ID	Requirement
MEM-001	Program memory **MUST** be organised in 8-bit words, byte-addressed.
MEM-002	The program counter PC **MUST** be 16 bits wide, giving a logical program address space of 64 KiB (0x0000–0xFFFF).
MEM-003	The physical program memory **MUST** occupy addresses 0x0000 to PMEM_SIZE − 1, where PMEM_SIZE is a synthesis parameter (default: 8 KiB). An instruction fetch or LPM read from an address ≥ PMEM_SIZE **MUST** return 0xFF (decoded as TRAP, see ISA-050) and **MUST NOT** alias to a lower address. The core **MUST** perform this check itself, independently of the memory connected to P_ADDR/P_DATA.
MEM-004	Program memory **MUST** be read-only for the processor. Its contents **MUST** be loadable from a file (e.g. .mem) for synthesis and simulation.
MEM-005	The processor **MUST** provide the LPM instruction (read a constant byte from program memory through a register pair) to support constant tables.

2.3 Data memory

ID	Requirement
MEM-010	Data memory **MUST** be organised in 8-bit words, byte-addressed, with a 16-bit address (64 KiB address space).
MEM-011	The physical data memory **MUST** have a fixed size of 2 KiB, mapped at 0x0000–0x07FF. A read from 0x0800–0xFFFF **MUST** return 0x00; a write to that range **MUST** be ignored.
MEM-012	Data memory **MUST** support a read or write of one byte per cycle.
MEM-013	Direct addressing **MUST** cover the whole data address space (16-bit address in the instruction); indirect addressing **MUST** use a register pair as the pointer.
MEM-014	The stack **MUST** reside in data memory, grow downwards and be addressed by the 16-bit register SP.
MEM-015	Special function registers (SFR) **MUST** reside in a separate 8-bit address space (256 addresses), independent of data memory (see Section 4).

3 Windowed register file

The 64 physical registers P0–P63 form 16 blocks of 4 registers (B0–B15). The program sees 8 of them as R0–R7. Because the window moves by 4 registers, adjacent windows share 4 registers, which are used to pass arguments and results between functions.

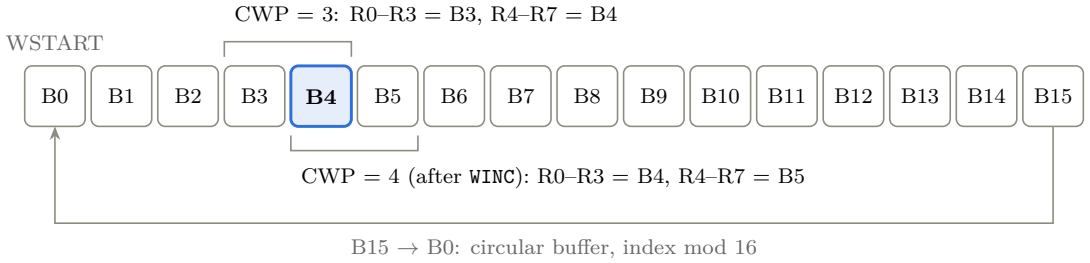
3.1 Mapping

Logical register R_n ($n = 0 \ldots 7$) in window CWP maps to the physical register

$$P_{idx} = (4 \cdot \mathrm{CWP} + n) \bmod 64 \tag{1}$$

Example: with CWP = 3, registers R0–R3 are P12–P15 (block B3) and R4–R7 are P16–P19 (block B4). After WINC (CWP = 4), the former R4–R7 become R0–R3 (Figure 1).

Each window occupies two consecutive blocks, so WINC allocates one new block while the other stays shared with the caller's window.



B4 is shared: caller's R4–R7 = callee's R0–R3 (arguments and results)
WINC with (CWP + 2) mod 16 = WSTART → WOVF exception, oldest block is spilled

Figure 1: Register windows: 16 blocks of 4 registers, adjacent windows share one block

3.2 Calling convention

1. The caller places up to 4 arguments in R4–R7 and executes `CALLW` (= `WINC` + `CALL`).
2. The callee sees the arguments in R0–R3; R4–R7 are its working registers and, at the same time, the argument area for further calls.
3. The callee returns results in R0–R3 and executes `RETW` (= `WDEC` + `RET`); the caller reads them in R4–R7.
4. More than 4 arguments are passed on the stack.

3.3 Requirements

ID	Requirement
REG-001	The processor **MUST** have 64 general-purpose 8-bit registers (P0–P63).
REG-002	At any time, 8 registers R0–R7 **MUST** be visible, mapped according to Equation (1).
REG-003	The window pointer CWP **MUST** be 4 bits wide (16 positions) and **MUST** be updated modulo 16.
REG-004	The `WINC` instruction **MUST** increment CWP by 1 (the window moves up by 4 registers); `WDEC` **MUST** decrement it by 1.
REG-005	The register file **MUST** support two reads and one write in a single cycle.
REG-006	Register pairs (R0:R1, R2:R3, R4:R5, R6:R7) **MUST** be usable as 16-bit address pointers, with the low byte in the even register.
REG-007	The register WSTART (4 bits) **MUST** point to the oldest window whose contents are still held in the register file.
REG-008	Executing `WINC` when (CWP + 2) mod 16 = WSTART **MUST** raise the window overflow exception (WOVF) before CWP is changed. This allows up to 15 simultaneously live windows (14 nested calls) without spilling to memory.
REG-009	Executing `WDEC` when CWP = WSTART **MUST** raise the window underflow exception (WUNF) before CWP is changed.
REG-010	After a WOVF/WUNF handler returns (`RETI`), the instruction that raised the exception **MUST** be executed again.
REG-011	The exception handler **MUST** be able to access any physical register through the SFRs PRIDX (index 0–63) and PRDAT (data), in order to save the WSTART block to the stack (spill) or restore it (fill).
REG-012	Register contents after reset **MAY** be undefined; the implementation **SHOULD** clear them in simulation to simplify debugging.

4 Special function registers and flags

Control and peripheral registers reside in the 8-bit SFR address space, which is reachable through the SFR addressing mode of every two-operand instruction (e.g. `OR R1, [SFR:I2CCON]`). Peripherals are therefore operated with the same instructions as memory.

4.1 SFR map

Address	Name	Access	Reset	Description
`0x00`	SR	R/W	`0x00`	Status register (flags, IE), see 4.2
`0x01`	SPL	R/W	see `INT-011`	Stack pointer, low byte
`0x02`	SPH	R/W	see `INT-011`	Stack pointer, high byte
`0x03`	CWP	R/W	`0x00`	Bits 3:0: current register window
`0x04`	WSTART	R/W	`0x00`	Bits 3:0: oldest window held in the register file
`0x05`	PRIDX	R/W	`0x00`	Bits 5:0: physical register index
`0x06`	PRDAT	R/W	–	Read/write of physical register P[PRIDX]
`0x08`	INTEN	R/W	`0x00`	Maskable interrupt enable (bit 0 INT0, bit 1 I^2C)
`0x09`	INTFLG	R/W1C	`0x00`	Pending interrupt flags; writing 1 clears
`0x10`	I2CCON	R/W	`0x00`	I^2C controller control, see Section 6
`0x11`	I2CSTA	R/W1C	`0x00`	I^2C controller status
`0x12`	I2CADR	R/W	`0x00`	Bits 6:0: 7-bit slave address
`0x13`	I2CTXD	W	`0x00`	Byte to transmit
`0x14`	I2CRXD	R	`0x00`	Last received byte
`0x15`	I2CBRG	R/W	`0xFF`	SCL clock divider
`0x07`, `0x0A`–`0x0F`, `0x16`–`0xFF`				Reserved; reads return `0x00`, writes are ignored

4.2 Status register SR

Bit	Name	Meaning
0	C	Carry / borrow; for I^2C instructions: transaction error
1	Z	Result is zero
2	N	Bit 7 of the result (sign)
3	V	Signed (two's complement) overflow
4–6	–	Reserved, read as 0
7	IE	Global enable for maskable interrupts

4.3 Requirements

ID	Requirement
`SFR-001`	The SFR address space **MUST** contain 256 addresses (8 bits) and **MUST** be separate from data memory.
`SFR-002`	The SFRs **MUST** be implemented as listed in the SFR map, including their reset values.
`SFR-003`	A read from a reserved address **MUST** return `0x00`; a write to it **MUST** be ignored without raising an exception.

ID	Requirement (cont.)
SFR-004	Bits marked R/W1C **MUST** be cleared by writing 1; writing 0 **MUST** leave them unchanged.
SFR-005	SPL/SPH **SHOULD** be written in the order SPL, SPH; the implementation **MAY** block interrupts for one instruction after a write to SPL to avoid an inconsistent SP.
SFR-006	ALU instructions **MUST** update the flags as listed in the "Flags" column of the instruction tables (Section 5); all other instructions **MUST NOT** change the flags unless stated otherwise.

5 Instruction set architecture (ISA)

Instructions are 1–4 bytes long: an opcode byte, an optional operand specifier byte and 0–2 extension bytes (immediate, displacement or address). The CISC core consists of 11 two-operand operations, each available in 7 addressing modes, including memory as the destination.

5.1 Instruction formats

Format	Byte 0	Byte 1	Bytes 2–3	Used for
A	opcode	–	–	No operands (`NOP`, `RET`, `EI`, ...)
B	opcode	specifier	–	Register / register pair
C	opcode	specifier	imm8 / disp8 / addr8	Immediate, displacement, SFR address
D	opcode	specifier	addr16 (LSB, MSB)	Direct address
E	opcode	rel8 / addr16 LSB	addr16 MSB	Jumps and calls (2 B with rel8, 3 B with addr16)

For the two-operand group (opcodes `0x00`–`0x7F`) the opcode byte holds the operation number OP in bits 7:3 and the addressing mode MODE in bits 2:0. The specifier byte holds Rd in bits 7:5, Rs in bits 4:2 (for indirect modes: the even number of the register pair), the direction bit D in bit 1 (0: result in Rd, 1: result in operand M) and a reserved 0 in bit 0 (Figure 2).



Byte 0: opcode

OP: 0–10 used, 11–15 reserved
MODE: see the addressing modes table

Byte 1: operand specifier

D = 0: result in Rd
D = 1: result in operand M

Bytes 2–3 (optional, depending on MODE): imm8 / disp8 / addr8 / addr16 (LSB first)

Figure 2: Encoding of a two-operand instruction (opcode = OP · 8 + MODE)

5.2 Addressing modes

MODE	Name	Operand M	D=1 allowed	Length
000	Register	Rs	no	2 B
001	Immediate	imm8	no	3 B
010	Direct	mem[addr16]	yes	4 B
011	Indirect	mem[Rs+1:Rs]	yes	2 B
100	Indirect post-increment	mem[pair], then pair ← pair + 1	yes	2 B
101	–	Reserved (ILL exception)	–	–
110	Indexed	mem[pair + signed disp8]	yes	3 B
111	SFR	sfr[addr8]	yes	3 B

5.3 Two-operand operations

OP	Mnemonic	Operation (D=0)	Flags
0	`MOV`	Rd ← M	Z, N
1	`ADD`	Rd ← Rd + M	C, Z, N, V
2	`ADC`	Rd ← Rd + M + C	C, Z, N, V
3	`SUB`	Rd ← Rd − M	C, Z, N, V
4	`SBC`	Rd ← Rd − M − C	C, Z, N, V
5	`AND`	Rd ← Rd ∧ M	Z, N; V = 0
6	`OR`	Rd ← Rd ∨ M	Z, N; V = 0
7	`XOR`	Rd ← Rd ⊕ M	Z, N; V = 0
8	`CMP`	Rd − M, flags only	C, Z, N, V
9	`TST`	Rd ∧ M, flags only	Z, N; V = 0
10	`MUL`	Rd+1:Rd ← Rd × M (unsigned); Rd even, D = 0	Z (16-bit result), C = high byte ≠ 0
11–15	–	Reserved (ILL exception)	–

With D=1 the result is written to M; for example `ADD [0x0420], R3` adds R3 to a memory location (read–modify–write in a single instruction). For `MOV` with D=1: M ← Rd.

5.4 Other instructions

Opcode	Mnemonic	Format	Operation
`0x80`–`0x89`	`INC, DEC, NEG, NOT, CLR, SHL, SHR, ASR, RLC, RRC`	B (Rd)	Single-operand operations on Rd
`0x8D` / `0x8E`	`PUSH Rd` / `POP Rd`	B	SP ← SP−1, mem[SP] ← Rd / Rd ← mem[SP], SP ← SP+1
`0x8F` / `0x90`	`INCW` / `DECW`	B (pair)	16-bit increment / decrement of a pair, flag Z
`0x91`	`LPM Rd, [pair]`	B	Rd ← program memory[pair] (`MEM-005`)
`0xB0`–`0xB5`	`JZ, JNZ, JC, JNC, JN, JNN`	E (rel8)	Conditional relative jump, −128 … + 127

Opcode	Mnemonic	Format	Operation
0xB9	JMP addr16	E	Unconditional jump
0xBB	CALL addr16	E	Push PC and jump
0xBD	DJNZ Rd, rel8	C	Rd ← Rd − 1; jump if Rd ≠ 0
0xBE	CALLW addr16	E	WINC, then CALL
0xC0 / 0xC1 / 0xC2	RET / RETI / RETW	A	Return / return from interrupt / WDEC + RET
0xC3 / 0xC4	WINC / WDEC	A	Move window by +4 / −4 registers
0xC5 / 0xC6	PUSHF / POPF	A	Push / pop SR
0xC7 / 0xC8	EI / DI	A	IE ← 1 / IE ← 0
0xC9	HALT	A	Sleep until interrupt
0xD0	I2CW Ra, Rd	B	Write byte Rd to slave at address Ra (Section 6.3)
0xD1	I2CR Rd, Ra	B	Read one byte from slave Ra into Rd
0xD2	I2CRR Rd, Ra	B	Register read: Ra = slave address, Ra+1 = register number (repeated START)
0xD3	I2CRRN [pair], Ra, #n	C (imm8 = n)	Read n bytes starting at register Ra+1 of slave Ra into data memory at [pair], pair ← pair + n
0xD4	I2CWN [pair], Ra, #n	C (imm8 = n)	Write n bytes from data memory at [pair] to slave Ra, pair ← pair + n
0xF0	NOP	A	No operation
0xFF	TRAP	A	Software exception (ILL vector); 0xFF = erased memory
others	–	–	Reserved, ILL exception

5.5 Requirements

ID	Requirement
ISA-001	The processor **MUST** implement all instructions listed in the instruction tables, with the specified encoding.
ISA-002	Every two-operand operation **MUST** work in all 7 addressing modes; MODE = 101 is reserved and **MUST** raise the ILL exception.
ISA-003	D=1 combined with the register or immediate mode, MUL with D=1 or an odd Rd, and an indirect mode with an odd Rs **MUST** raise the ILL exception.
ISA-004	Instructions with D=1 **MUST** perform the read–modify–write of the memory operand as an atomic operation (no interrupt is taken in between).
ISA-005	Relative jumps **MUST** compute the target relative to the address of the next instruction.
ISA-006	CALL **MUST** push the 16-bit return address onto the stack (high byte first, so that the low byte ends up at the lower address).
ISA-007	The number of clock cycles for every instruction and addressing mode **MUST** be documented in the appendix “Instruction timing table”.
ISA-040	The instructions I2CW, I2CR, I2CRR, I2CRRN and I2CWN **MUST** stall execution until the transaction completes or times out, and **MUST** set C = 1 on error (NACK, arbitration lost, timeout).

ID	Requirement (cont.)
ISA-041	For `I2CRRN` and `I2CWN` the pair **MUST** be given by its even register number in the Rd field and Ra in the Rs field; an odd pair number or n = 0 **MUST** raise the ILL exception. After the instruction the pair **MUST** point just past the last byte actually transferred (also when the transaction is aborted by an error).
ISA-050	Opcode `0xFF` and every reserved opcode **MUST** raise the ILL exception; the saved PC points to that instruction.

6 I²C controller

The I²C controller operates as a bus master (7-bit addressing, 100 and 400 kHz) and offers two levels of use: six SFRs for full control of a transaction, and five high-level instructions (`I2CW`, `I2CR`, `I2CRR`, and the multi-byte `I2CRRN`, `I2CWN`) that cover the common cases in a single line of code.

6.1 Rationale for the register set

Three registers alone (slave address, byte to send, received byte) are enough to write or read a single byte, but they do not allow:

- reading a sensor register, which requires writing the register number followed by a **repeated START** without a STOP;
- multi-byte transactions (e.g. an EEPROM page write, reading 6 bytes from an accelerometer);
- error detection: missing ACK (device not responding), arbitration lost, a stuck bus;
- setting the SCL frequency and checking whether a transfer has finished.

The three registers are therefore kept (I2CADR, I2CTXD, I2CRXD), and a control, a status and a clock divider register are added.

6.2 Registers

I2CCON (`0x10`)

Bit	Name	Meaning
7	EN	Controller enable
6	IE	Interrupt when an operation completes (DONE)
5	START	Write 1: START (or repeated START if the bus is already owned) followed by the address byte {I2CADR, RW}; cleared by hardware
4	STOP	Write 1: STOP condition; cleared by hardware
3	RD	Write 1: receive one byte into I2CRXD; cleared by hardware
2	ACKS	Acknowledge sent after a received byte: 0 = ACK, 1 = NACK (last byte)
1	RW	Direction in the address phase: 0 = write, 1 = read
0	–	Reserved

I2CSTA (`0x11`)

Bit	Name	Meaning
0	TIP	Byte transfer in progress
1	DONE	Operation complete (R/W1C); interrupt source
2	NACK	The slave did not acknowledge the last byte
3	ARBL	Arbitration lost (R/W1C)
4	BB	Bus busy (between START and STOP)
5	RXRDY	New byte in I2CRXD; cleared by reading I2CRXD
6	TOUT	Timeout expired (R/W1C)
7	–	Reserved

Writing I2CTXD while BB = 1 starts transmission of that byte. The SCL frequency is

$$f_{\mathrm{SCL}} = \frac{f_{\mathrm{CLK}}}{4 \cdot (\mathrm{I2CBRG} + 1)} \tag{2}$$

Example for $f_{\mathrm{CLK}} = 16\,\mathrm{MHz}$: I2CBRG = 39 gives 100 kHz, I2CBRG = 9 gives 400 kHz.

6.3 High-level instructions

Instruction	Bus sequence
`I2CW Ra, Rd`	START, address(Ra)+W, byte Rd, STOP
`I2CR Rd, Ra`	START, address(Ra)+R, byte → Rd (NACK), STOP
`I2CRR Rd, Ra`	START, address(Ra)+W, byte Ra+1, repeated START, address(Ra)+R, byte → Rd (NACK), STOP
`I2CRRN [pair], Ra, #n`	START, address(Ra)+W, byte Ra+1, repeated START, address(Ra)+R, n bytes → mem[pair… pair+n−1] (ACK after each byte, NACK after the last), STOP
`I2CWN [pair], Ra, #n`	START, address(Ra)+W, n bytes from mem[pair… pair+n−1], STOP

The multi-byte instructions transfer data directly between the I²C controller and data memory; the bytes are stored in bus order at increasing addresses. To write device registers with `I2CWN`, the first byte of the buffer holds the register number. Because the processor stalls for the whole transaction (about 9 SCL periods per byte: roughly 0.8 ms for 6 bytes at 100 kHz), longer transfers **SHOULD** use 400 kHz.

Example: reading 6 bytes (registers `0x3B`–`0x40`) from an accelerometer at address `0x68` into a buffer:

```
    MOV    R2, #0x68        ; slave address
    MOV    R3, #0x3B        ; first register
    MOV    R4, #<buf        ; R5:R4 = buffer address
    MOV    R5, #>buf
    I2CRRN [R4], R2, #6     ; buf[0..5] <- registers 0x3B..0x40
    JC     i2c_error
```

The repeated START (Sr) keeps ownership of the bus between writing the register number and reading the data, so no other master can intervene (Figure 3).

Example: reading register `0x75` from a sensor at address `0x68`:

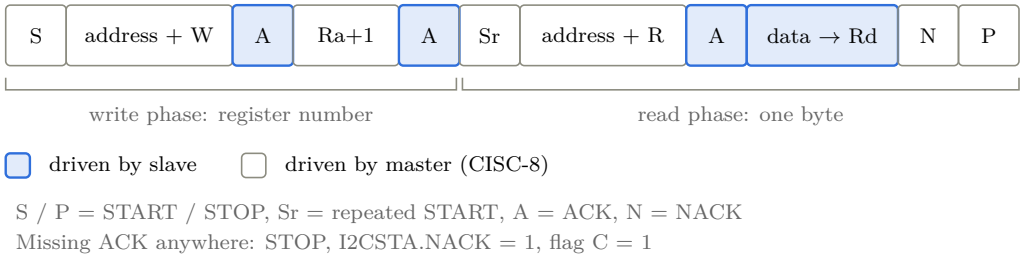


Figure 3: I²C frame generated by `I2CRR`: register number write, repeated START, single-byte read

```
    MOV   R2, #0x68      ; slave address
    MOV   R3, #0x75      ; register number
    I2CRR R0, R2         ; R0 <- data read
    JC    i2c_error      ; C = 1: NACK / timeout
```

6.4 Requirements

ID	Requirement
I2C-001	The controller **MUST** operate as a master only, with 7-bit addressing, in accordance with NXP UM10204. Slave mode and 10-bit addressing are not supported.
I2C-002	The controller **MUST** support Standard mode (100 kHz) and Fast mode (400 kHz); the frequency is set by I2CBRG according to Equation (2).
I2C-003	The controller **MUST** generate START, repeated START and STOP conditions, and transmit and receive bytes with ACK/NACK handling.
I2C-004	The controller **MUST** support clock stretching by the slave by sampling the SCL line.
I2C-005	The controller **MUST** detect arbitration loss (SDA read back differs from SDA driven), release the bus and set ARBL.
I2C-006	The controller **MUST** abort the operation and set TOUT when SCL is held low longer than the timeout (synthesis parameter, default 25 ms).
I2C-007	SDA and SCL **MUST** be driven as open-drain (low or high-impedance only); the pull-up resistors are external.
I2C-008	The SDA and SCL inputs **MUST** be synchronised to CLK (at least 2 flip-flops) and **SHOULD** have a glitch filter suppressing pulses shorter than 50 ns.
I2C-009	Setting DONE while IE = 1 **MUST** raise the I²C interrupt.
I2C-010	The instructions `I2CW`, `I2CR`, `I2CRR`, `I2CRRN` and `I2CWN` **MUST** use the same controller and leave the final status in I2CSTA.
I2C-011	For `I2CRRN` and `I2CWN` the controller **MUST** read and write data memory directly, one byte per transferred I²C byte (`ISA-041`).

7 Interrupts, exceptions and reset

The processor has three non-maskable exceptions (WOVF, WUNF, ILL) and two maskable interrupts (INT0, I²C), served through a fixed vector table at the start of program memory; the user program starts at `0x0020`.

7.1 Vector table

Address	Source	Type	Priority
`0x0000`	RESET	–	highest
`0x0004`	WOVF: register window overflow	exception, non-maskable	1
`0x0008`	WUNF: register window underflow	exception, non-maskable	2
`0x000C`	ILL: illegal instruction / `TRAP`	exception, non-maskable	3
`0x0010`	INT0: external interrupt	maskable	4
`0x0014`	I^2C: operation complete	maskable	5
`0x0018`–`0x001F`	Reserved	–	–

Each vector is 4 bytes long, which fits a `JMP addr16` instruction (3 bytes).

7.2 Requirements

ID	Requirement
`INT-001`	A maskable interrupt **MUST** be taken only when SR.IE = 1, its bit in INTEN is set and its flag in INTFLG is set.
`INT-002`	Interrupts **MUST** be taken only between instructions; exceptions (WOVF, WUNF, ILL) are raised before execution of the instruction that caused them.
`INT-003`	On entry to a handler, the processor **MUST** push PC (2 bytes) and SR (1 byte) onto the stack, clear SR.IE and jump to the vector address.
`INT-004`	For an interrupt, the saved PC **MUST** point to the next instruction; for an exception, to the instruction that raised it.
`INT-005`	`RETI` **MUST** pop SR and PC from the stack (restoring IE).
`INT-006`	When several sources are pending, the source with the highest priority according to the vector table **MUST** be served first.
`INT-007`	The latency from the end of the current instruction to the first instruction of the handler **SHOULD** not exceed 8 cycles; the actual value **MUST** be documented.
`INT-008`	The INT0 input **MUST** be synchronised to CLK; the interrupt is triggered on the falling edge.
`INT-009`	A handler that uses `WINC` **MUST** execute `WDEC` before `RETI`; the WOVF/WUNF handlers **MUST NOT** use `WINC` or `WDEC`.
`INT-010`	`HALT` **MUST** suspend execution until an interrupt occurs (also with IE = 0, in which case execution resumes at the next instruction without entering the handler).
`INT-011`	After reset: PC = `0x0000`, SR = `0x00`, CWP = WSTART = 0, INTEN = INTFLG = 0, I^2C controller disabled, SP = `0x0800` (the first `PUSH` writes the last byte of data memory, `0x07FF`).
`INT-012`	The `RST_N` signal **MUST** be active low, with asynchronous assertion and synchronous de-assertion (2 flip-flops).

8 External interfaces and non-functional requirements

The core exposes the clock, reset, one interrupt input, the two I^2C lines and the buses of both memories, so that the memories can be either FPGA block RAM or external devices.

8.1 Core signals

Signal	Direction	Width	Description
`CLK`	in	1	System clock
`RST_N`	in	1	Reset, active low (`INT-012`)
`INT0_N`	in	1	External interrupt, falling edge
`SDA`	in/out	1	I²C data, open-drain
`SCL`	in/out	1	I²C clock, open-drain
`P_ADDR`	out	16	Program memory address
`P_DATA`	in	8	Program memory data
`D_ADDR`	out	16	Data memory address
`D_WDATA`	out	8	Write data
`D_RDATA`	in	8	Read data
`D_WE`	out	1	Write enable
`HALTED`	out	1	Processor in HALT state (diagnostics)

8.2 Interface requirements

ID	Requirement
`IO-001`	The core **MUST** provide the signals listed in the core signal table.
`IO-002`	The memory buses **MUST** be synchronous with a read latency of 1 cycle (compatible with block RAM of typical FPGAs).
`IO-003`	A top-level variant with embedded memories and tri-state SDA/SCL drivers **MUST** exist, ready for synthesis on the target board.

8.3 Non-functional requirements

ID	Requirement
`NF-001`	The design **MUST** be described in a synthesisable HDL (Verilog/SystemVerilog or VHDL – to be decided).
`NF-002`	The design **MUST** be fully synchronous: no latches, no gated clocks and no combinational loops; it **MUST** pass lint without critical warnings.
`NF-003`	The target clock frequency **SHOULD** be at least 50 MHz on the selected FPGA (to be confirmed once the device is chosen).
`NF-004`	The I²C timeout and the default value of I2CBRG **MUST** be parameters (generic/parameter) of the top-level module.
`NF-005`	Each block (ALU, register file, control unit, I²C controller, interrupt controller) **MUST** be a separate module with its own testbench.
`NF-006`	The source code **MUST** be kept under version control.
`NF-007`	The documentation **MUST** include this specification, a microarchitecture description and the instruction timing table.